Error handling patterns

Module 3 - Lesson 3

Overview

Errors are part of running code. Python’s exceptions let you anticipate failure, respond to it, and clean up resources, instead of letting the program crash.

Key Concepts

- try/except catches exceptions; except Exception as e names the error.
- Match the exception type: ValueError for bad values, KeyError for missing keys, etc.
- else runs only when no exception occurred; finally always runs for cleanup.
- Raise your own errors with raise ValueError("message") and custom exception classes.
- Don’t catch and silence errors; log, translate, or re-raise.
- EAFP (ask for forgiveness) vs LBYL (look before you leap) are the two styles.

Example

```
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

try:
    result = divide(10, 0)
except ValueError as err:
    print("Invalid input:", err)
finally:
    print("Done")
```

Summary

Exceptions communicate failure and let you handle it gracefully. Precise except clauses, finally for cleanup, and raising clear errors keep programs predictable.

Practice Checklist

- Wrap a fragile operation in try/except and print a friendly message.
- Raise a custom exception and catch it by type.
- Use finally to close a file or connection on success and failure.